

TECHNICAL OVERVIEW & DEPLOYMENT GUIDE

Alteration App

Architecture, API, and Production Deployment

Prepared for: Management Review

Document version: 1.0

Date: 15 May 2026

Author: Engineering team

Executive summary

The **Alteration App** is a standalone web application that adds alteration-tracking capability to our existing **QuickBill** product. It lets staff pick an existing bill, record alteration details (quantity, remarks, delivery date, status), and review reports.

Key characteristics:

- **No new database.** It reads and writes QuickBill's existing SQL Server database directly, so bills saved in QuickBill appear here instantly with no sync layer or duplication of truth.
- **Single static binary + static web bundle.** Easy to deploy and operate; no runtime dependencies on the production host beyond a stock Linux + nginx.
- **Two clear layers** — a Go HTTP API and a React (TypeScript) single-page application — connected through a thin reverse-proxy.

Table of contents

1. What the application does
2. Architecture
3. Technology stack
4. How the API works
5. How the frontend works
6. Configuration
7. Building for production
8. Deploying on Linux with nginx
9. Operations & maintenance
10. Security & hardening
11. Summary for management

1. What the application does

End-users (shop staff) can:

1. **Pick a bill** from a list of vouchers eligible for alteration.
2. **Record alteration items** — for each line item: how many pieces to alter, remarks (e.g., "shorten sleeve by 1 inch"), and a delivery date.
3. **Track status** of each item and of the overall alteration (Received, In Progress, Ready, Delivered, etc.).

4. **View reports** of alterations over a chosen period.

Because the app talks directly to QuickBill's database, no manual data entry is duplicated and there is no risk of the two systems drifting out of sync.

2. Architecture



Key design decision: the app does not own a database. It uses QuickBill's SQL Server tenant database, with three migration scripts that add the alteration tables (`QbVoucherAlteration` , `QbVoucherAlterationItems`) and seed a voucher type. This guarantees data consistency with QuickBill at all times.

3. Technology stack

Layer	Technology	Why
Backend	Go 1.26 (standard <code>net/http</code> mux)	Single static binary; fast, low-memory, no JVM
DB driver	github.com/microsoft/go-mssqldb	Official Microsoft driver for SQL Server
Config	github.com/joho/godotenv/autoload	Loads <code>.env</code> automatically
Frontend	React 19 + TypeScript	Type-safe, modern UI
Build tool	Vite 8	Fast dev server, optimised production build
Styling	Tailwind CSS 4	Consistent design system, small CSS bundle
Routing	React Router 7	Client-side navigation
Database	Microsoft SQL Server (QuickBill's existing DB)	No new infrastructure required
Web server	nginx (Linux production)	Reverse proxy + TLS + static file serving

4. How the API works

4.1 Entry point

The HTTP server is a single Go file: `cmd/server/main.go`. On startup it:

1. Loads environment variables from `.env`.
2. Opens a verified connection pool to SQL Server (25 max open, 25 idle, 5-minute connection lifetime).
3. Registers route handlers on Go's standard library mux using method-prefixed patterns (e.g. `GET /api/...`).
4. Listens on the port from `PORT` (default `8080`).

Graceful degradation: if the database is unreachable at startup, the server still starts and every API endpoint returns HTTP `503 Service Unavailable` with a clear message. This means deployment never blocks on database availability.

4.2 Endpoints

Method	Path	Purpose
GET	<code>/api/vouchers/for-alteration</code>	List bills available to alter (bill picker)
GET	<code>/api/voucher/{qbguid}/alteration</code>	Load one voucher and any existing alteration
POST	<code>/api/voucher/{qbguid}/alteration</code>	Create / update an alteration with items
PATCH	<code>/api/alterations/{qbguid}/status</code>	Change overall alteration status
PATCH	<code>/api/alterations/{qbguid}/items/{itemQbguid}/status</code>	Change one item's status
GET	<code>/api/alterations/reports</code>	Reports listing

All responses are JSON. Errors follow a consistent shape:

```
{
  "error": "alterationQty 12 exceeds DocQty 5",
  "status": 400,
  "detail": { "voucherItemGuid": "...", "alterationQty": 12, "docQty": 5 }
}
```

4.3 Request lifecycle (example — saving an alteration)

The save handler (`POST /api/voucher/{qbguid}/alteration`) illustrates the pattern used throughout:

1. **Parse** path parameter and decode JSON body into a typed struct.
2. **Validate** input — required fields, max lengths, qty > 0, valid status.
3. **Begin a database transaction.** All writes happen inside this transaction so partial failures are impossible.
4. **Business rule checks** — voucher exists, every item belongs to that voucher, requested quantity does not exceed the document quantity.
5. **Insert or update** — if an alteration already exists for the voucher, soft-delete the previous detail rows (`ActiveFlag = 0`) and increment `AlterId` , then re-insert. If new, allocate a voucher number from the QuickBill counter and insert header + items.
6. **Sync header status** to the minimum item status (e.g. if any item is still “Received”, the header is “Received”).
7. **Commit** and return JSON with the new identifiers and status.

Typed errors map cleanly to HTTP status codes — voucher not found → `404` , validation errors → `400` , infrastructure errors → `500` .

5. How the frontend works

- The React app is a **single-page application** built with Vite.
- Two pages are routed client-side:
 - `/` — bill picker
 - `/alteration` — alteration form
- All API calls are made to relative paths beginning with `/api/...`. The frontend never needs to know the backend's hostname or port.
- During development, Vite proxies `/api/*` to `http://localhost:8080`.
- In production, nginx performs the same proxy. The browser sees a single origin, which avoids CORS complexity and works cleanly with HTTPS.

6. Configuration

All runtime configuration lives in a single `.env` file at the project root. The file is git-ignored. Two variables are used:

Variable	Example	Purpose
<code>PORT</code>	<code>8080</code>	HTTP port the Go server listens on
<code>DATABASE_URL</code>	<code>sqlserver://app_user:STRONGPASS@10.0.0.5:1433?database=QbAppDb1&encrypt=true</code>	SQL Server DSN

For production the DB user should be a **dedicated, least-privilege account** with read/write on the alteration tables and read on the QuickBill voucher tables it touches — never the `sa` account.

7. Building for production

```
# 1. Build the React frontend → web/dist/ (static HTML/CSS/JS)
cd web
npm install
npm run build
cd ..

# 2. Build the Go API binary
go build -o alteration ./cmd/server

# Cross-compile from Windows to Linux:
$env:GOOS = "linux"
$env:GOARCH = "amd64"
go build -o alteration ./cmd/server
```

This produces two deployable artifacts:

- `alteration` — a single static Linux binary (no runtime dependencies).
- `web/dist/` — a folder of static files.

8. Deploying on Linux with nginx

8.1 Prepare the server

```
sudo apt update
sudo apt install -y nginx
sudo adduser --system --no-create-home --group alteration
sudo mkdir -p /opt/alteration /var/www/alteration
```

8.2 Copy artifacts to the server

```
# From build machine
scp alteration          user@server:/tmp/
scp -r web/dist/*      user@server:/tmp/dist/
scp .env               user@server:/tmp/

# On the server
sudo mv /tmp/alteration /opt/alteration/alteration
sudo mv /tmp/.env       /opt/alteration/.env
sudo mv /tmp/dist/*     /var/www/alteration/
sudo chown -R alteration:alteration /opt/alteration
sudo chmod 600 /opt/alteration/.env
sudo chmod +x /opt/alteration/alteration
```

8.3 Run the API as a systemd service

Create `/etc/systemd/system/alteration.service` :

```
[Unit]
Description=Alteration App API
After=network.target

[Service]
Type=simple
User=alteration
WorkingDirectory=/opt/alteration
EnvironmentFile=/opt/alteration/.env
ExecStart=/opt/alteration/alteration
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

Enable and start it:

```
sudo systemctl daemon-reload
sudo systemctl enable --now alteration
sudo systemctl status alteration
sudo journalctl -u alteration -f
```

8.4 Configure nginx

Create `/etc/nginx/sites-available/alteration` :


```

server {
    listen 80;
    server_name alteration.yourcompany.com;

    root /var/www/alteration;
    index index.html;

    # Single-page app fallback: unknown paths return index.html
    location / {
        try_files $uri /index.html;
    }

    # Reverse proxy /api to the Go binary
    location /api/ {
        proxy_pass          http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header    Host            $host;
        proxy_set_header    X-Real-IP       $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_read_timeout  60s;
    }

    # Aggressive cache for static assets
    location ~* \.(js|css|svg|png|jpg|woff2)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}

```

Enable and reload:

```

sudo ln -s /etc/nginx/sites-available/alteration /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

```

8.5 Enable HTTPS

```

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d alteration.yourcompany.com

```

Certbot installs an auto-renewal timer; no further action is required.

8.6 Firewall and database connectivity

- Open only ports `80` and `443` to the internet (`ufw allow 80/tcp` , `ufw allow 443/tcp`).
- Do **not** expose port `8080` externally — it is bound to localhost and reached only by nginx.

- On the SQL Server host, open port `1433` (or the named-instance port) *only* to the Linux app server's IP.

9. Operations & maintenance

Task	Command
View live API logs	<code>sudo journalctl -u alteration -f</code>
Restart the API (after new binary)	<code>sudo systemctl restart alteration</code>
Reload nginx (after new frontend bundle)	<code>sudo systemctl reload nginx</code>
Check API service health	<code>sudo systemctl status alteration</code>
Deploy a backend update	<code>scp</code> new binary → <code>systemctl restart</code>
Deploy a frontend update	<code>scp</code> new <code>dist/*</code> → <code>systemctl reload nginx</code>

Backend restarts are sub-second; nginx reloads are hitless. Both can be done during business hours with no perceptible disruption.

10. Security & hardening

- API runs as an unprivileged systemd user (`alteration`), not root.
- `.env` is `chmod 600` and owned by the service user — DB password is not world-readable.
- API binds to `127.0.0.1:8080` behind nginx; never directly exposed.
- TLS terminates at nginx via Let's Encrypt certificates with automatic renewal.
- The SQL Server account used by the app has only the privileges it needs (no `sa` , no DDL on production).
- All inputs are validated server-side; all writes are transactional.

11. Summary for management

1. **Low operational footprint.** One Go binary plus a folder of static files; stock Linux + nginx is enough to host it.
2. **No data duplication.** Uses QuickBill's existing SQL Server database directly — bills appear instantly and stay consistent.

3. **Reliable writes.** Every save is transactional; bad input is rejected with clear error messages; infrastructure issues degrade gracefully.
4. **Easy to update.** Restart of the backend is sub-second; nginx reload is hitless. Updates can be deployed during the working day.
5. **Secure by default.** Unprivileged service user, restricted file permissions, internal-only API port, HTTPS via Let's Encrypt, least-privilege database account.
6. **Industry-standard stack.** Go, React, nginx, SQL Server — easy to staff, easy to support, easy to audit.